

1st TASK

CODE:-

```
numbers = [64, 34, 25, 12, 22, 11, 90]
```

```
def bubble_sort(nums):
    arr = nums.copy()
    comparisons = 0
    swaps = 0
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1

    return arr, comparisons, swaps


def bubble_sort_optimized(nums):
    arr = nums.copy()
    comparisons = 0
    swaps = 0
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
                swapped = True

        # Early exit: if no swaps happened in this pass, the list is sorted
        if not swapped:
            break

    return arr, comparisons, swaps


def show_one_pass_demo():
    arr = [64, 34, 25, 12, 22, 11, 90]
    print("One complete pass of Bubble Sort:")
    print("Start:", arr)

    for j in range(0, len(arr) - 1):
        print(f"Compare {arr[j]} and {arr[j + 1]}")
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            print("Swap ->", arr)
        else:
            print("No swap ->", arr)
```

```

    print("After one pass:", arr)
    print()

def compare_versions(test_cases):
    print("=== Bubble Sort Comparison ===")
    for case in test_cases:
        original_sorted, original_comp, original_swaps = bubble_sort(case)
        improved_sorted, improved_comp, improved_swaps = bubble_sort_optimized(case)

        print(f"Input: {case}")
        print(f"Original: sorted={original_sorted}, comparisons={original_comp}, swaps={original_swaps}")
        print(f"Optimized: sorted={improved_sorted}, comparisons={improved_comp}, swaps={improved_swaps}")
        print("-" * 60)

# Student tasks
# 1) Implement Bubble Sort
sorted_numbers, comp_count, swap_count = bubble_sort(numbers)
print("Original problem input:", numbers)
print("Sorted list:", sorted_numbers)
print("Comparisons:", comp_count)
print("Swaps:", swap_count)
print()

# 2) Explain one complete pass of Bubble Sort
show_one_pass_demo()

# 3) Test at least three input lists
test_cases = [
    [64, 34, 25, 12, 22, 11, 90],
    [5, 1, 4, 2, 3],
    [9, 8, 7, 6, 5],
    [1, 2, 3, 4, 5]
]

compare_versions(test_cases)

# 4) Ask AI about early-exit optimization
print("AI note: Early-exit optimisation checks whether a pass made any swaps.")
print("If no swaps happen in a pass, the array is already sorted and the loop can stop early.")
print("This improves performance on nearly sorted data.")
print()

# 5) Compare original and improved versions
print("Comparison summary:")
print("The original Bubble Sort always makes all passes.")
print("The optimized version can stop early when the list becomes sorted, saving comparisons.")
print("This is most useful for already-sorted or nearly-sorted lists.")

```

OUTPUT:-

Original problem input: [64, 34, 25, 12, 22, 11, 90]

Sorted list: [11, 12, 22, 25, 34, 64, 90]

Comparisons: 21

Swaps: 14

One complete pass of Bubble Sort:

Start: [64, 34, 25, 12, 22, 11, 90]

Compare 64 and 34

Swap -> [34, 64, 25, 12, 22, 11, 90]

Compare 64 and 25

Swap -> [34, 25, 64, 12, 22, 11, 90]

Compare 64 and 12

Swap -> [34, 25, 12, 64, 22, 11, 90]

Compare 64 and 22

Swap -> [34, 25, 12, 22, 64, 11, 90]

Compare 64 and 11

Swap -> [34, 25, 12, 22, 11, 64, 90]

Compare 64 and 90

No swap -> [34, 25, 12, 22, 11, 64, 90]

After one pass: [34, 25, 12, 22, 11, 64, 90]

=== Bubble Sort Comparison ===

Input: [64, 34, 25, 12, 22, 11, 90]

Original: sorted=[11, 12, 22, 25, 34, 64, 90], comparisons=21, swaps=14

Optimized: sorted=[11, 12, 22, 25, 34, 64, 90], comparisons=21, swaps=14

Input: [5, 1, 4, 2, 3]

Original: sorted=[1, 2, 3, 4, 5], comparisons=10, swaps=6

Optimized: sorted=[1, 2, 3, 4, 5], comparisons=9, swaps=6

Input: [9, 8, 7, 6, 5]

Original: sorted=[5, 6, 7, 8, 9], comparisons=10, swaps=10

Optimized: sorted=[5, 6, 7, 8, 9], comparisons=10, swaps=10

Input: [1, 2, 3, 4, 5]

Original: sorted=[1, 2, 3, 4, 5], comparisons=10, swaps=0

Optimized: sorted=[1, 2, 3, 4, 5], comparisons=4, swaps=0

AI note: Early-exit optimisation checks whether a pass made any swaps.

If no swaps happen in a pass, the array is already sorted and the loop can stop early.

This improves performance on nearly sorted data.

Comparison summary:

The original Bubble Sort always makes all passes.

The optimized version can stop early when the list becomes sorted, saving comparisons.

This is most useful for already-sorted or nearly-sorted lists.

Task2

CODE:-

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
target = 45

def linear_search(nums, target_value):
    comparisons = 0

    for index, value in enumerate(nums):
        comparisons += 1
        if value == target_value:
            return index, comparisons, True

    return -1, comparisons, False

def binary_search(nums, target_value):
    left = 0
    right = len(nums) - 1
    comparisons = 0

    while left <= right:
        mid = (left + right) // 2
        comparisons += 1

        if nums[mid] == target_value:
            return mid, comparisons, True
        elif nums[mid] < target_value:
            left = mid + 1
        else:
            right = mid - 1
```

```

        return -1, comparisons, False

def test_searches():
    test_values = [5, 45, 89, 100] # first, middle, last,
    absent

    print("=== Linear Search vs Binary Search ===")
    for value in test_values:
        lin_index, lin_comp, lin_found =
linear_search(numbers, value)
        bin_index, bin_comp, bin_found =
binary_search(numbers, value)

        print(f"Target: {value}")
        print(f"Linear Search: index={lin_index},
comparisons={lin_comp}, found={lin_found}")
        print(f"Binary Search: index={bin_index},
comparisons={bin_comp}, found={bin_found}")
        print("-" * 50)

# Main program
print("Original numbers:", numbers)
print("Target:", target)
print()

# 1) Linear Search
linear_index, linear_comp, linear_found =
linear_search(numbers, target)
print("Linear Search Result:")
print(f"Index: {linear_index}")
print(f"Comparisons: {linear_comp}")

```

```
print(f"Found: {linear_found}")
print()

# 2) Binary Search
binary_index, binary_comp, binary_found =
binary_search(numbers, target)
print("Binary Search Result:")
print(f"Index: {binary_index}")
print(f"Comparisons: {binary_comp}")
print(f"Found: {binary_found}")
print()

# 3) Test first, middle, last, absent
test_searches()

# 4) Explain why Binary Search requires sorted data
print("AI note: Binary Search needs sorted data because it
repeatedly removes half of the list.")
print("If the list is not sorted, the middle element
cannot reliably tell us which half contains the target.")
print("So the algorithm depends on ordering to discard
incorrect sections.")
print()

# 5) Compare suitable use cases
print("Comparison summary:")
print("- Linear Search: simple and works on unsorted
lists, but checks each item one by one.")
print("- Binary Search: much faster on sorted lists, with
 $O(\log n)$  time, but requires sorted data.")
print("- Best use case: use Linear Search for small or
unsorted data; use Binary Search for large sorted data.")
print()
```

```
# 6) AI suggestion for an optimisation
print("AI optimisation suggestion:")
print("If the list is sorted and you search many times,
keep it sorted and use Binary Search.")
print("For repeated searches, avoid re-sorting each time
and use a sorted list or a database index.")
```

output:-

Original numbers: [5, 12, 18, 23, 31, 45, 56, 72, 89]

Target: 45

Linear Search Result:

Index: 5

Comparisons: 6

Found: True

Binary Search Result:

Index: 5

Comparisons: 3

Found: True

=== Linear Search vs Binary Search ===

Target: 5

Linear Search: index=0, comparisons=1, found=True

Binary Search: index=0, comparisons=3, found=True

Target: 45

Linear Search: index=5, comparisons=6, found=True

Binary Search: index=5, comparisons=3, found=True

Target: 89

Linear Search: index=8, comparisons=9, found=True

Binary Search: index=8, comparisons=4, found=True

Target: 100

Linear Search: index=-1, comparisons=9, found=False

Binary Search: index=-1, comparisons=4, found=False

AI note: Binary Search needs sorted data because it repeatedly removes half of the list.

If the list is not sorted, the middle element cannot reliably tell us which half contains the target.

So the algorithm depends on ordering to discard incorrect sections.

Comparison summary:

- Linear Search: simple and works on unsorted lists, but checks each item one by one.
- Binary Search: much faster on sorted lists, with $O(\log n)$ time, but requires sorted data.
- Best use case: use Linear Search for small or unsorted data; use Binary Search for large sorted data.

AI optimisation suggestion:

If the list is sorted and you search many times, keep it sorted and use Binary Search.

For repeated searches, avoid re-sorting each time and use a sorted list or a database index.Task
3

Code:-

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

# TODO:
# Sort using Bubble Sort, Selection Sort and Insertion Sort.
# Compare their performance.

def bubble_sort(nums):
    arr = nums.copy()
    comparisons = 0
    swaps = 0

    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1

    return arr, comparisons, swaps

def selection_sort(nums):
    arr = nums.copy()
    comparisons = 0
    swaps = 0

    n = len(arr)
    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            comparisons += 1
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            swaps += 1

    return arr, comparisons, swaps

def insertion_sort(nums):
    arr = nums.copy()
    comparisons = 0
    shifts = 0

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
```

```

        while j >= 0:
            comparisons += 1
            if arr[j] > key:
                arr[j + 1] = arr[j]
                shifts += 1
                j -= 1
            else:
                break

        arr[j + 1] = key

    return arr, comparisons, shifts

def compare_algorithms(input_list):
    bubble_result, bubble_comp, bubble_swaps = bubble_sort(input_list)
    selection_result, selection_comp, selection_swaps = selection_sort(input_list)
    insertion_result, insertion_comp, insertion_shifts = insertion_sort(input_list)

    print("=== Sorting Algorithm Comparison ===")
    print(f"Input list: {input_list}")
    print()

    print("Bubble Sort:")
    print(f"Sorted list: {bubble_result}")
    print(f"Comparisons: {bubble_comp}")
    print(f"Swaps: {bubble_swaps}")
    print("-" * 40)

    print("Selection Sort:")
    print(f"Sorted list: {selection_result}")
    print(f"Comparisons: {selection_comp}")
    print(f"Swaps: {selection_swaps}")
    print("-" * 40)

    print("Insertion Sort:")
    print(f"Sorted list: {insertion_result}")
    print(f"Comparisons: {insertion_comp}")
    print(f"Shifts: {insertion_shifts}")
    print("-" * 40)
    print()

def test_nearly_sorted_case():
    nearly_sorted = [1, 2, 3, 4, 5, 10, 6, 7, 8, 9]

    print("=== Nearly Sorted Data Test ===")
    print(f"Input list: {nearly_sorted}")
    print()

    for name, func in [
        ("Bubble Sort", bubble_sort),
        ("Selection Sort", selection_sort),
        ("Insertion Sort", insertion_sort)
    ]:
        result, comparisons, movement = func(nearly_sorted)
        print(f"{name}: sorted={result}, comparisons={comparisons}, movements={movement}")

```

```

    print("-" * 50)
    print()

# Main program
compare_algorithms(data)

# Additional test for nearly sorted data
test_nearly_sorted_case()

# AI notes
print("AI note: Bubble Sort is simple to understand and easy to implement, but it is usually slower than
the others.")
print("Selection Sort makes fewer swaps because it selects the minimum element and swaps only once per
pass.")
print("Insertion Sort is often the best choice for nearly sorted data because it can insert items with
very few shifts.")
print()

# Theoretical complexity comparison
print("Theoretical complexity:")
print("Bubble Sort:  $O(n^2)$  time,  $O(1)$  extra space")
print("Selection Sort:  $O(n^2)$  time,  $O(1)$  extra space")
print("Insertion Sort:  $O(n^2)$  time in the general case, but  $O(n)$  on nearly sorted data")
print("This means the experimental results should often match the quadratic nature of these algorithms,")
print("while Insertion Sort may perform significantly better on almost-sorted lists.")
print()

# Final comparison
print("Comparison summary:")
print("- Bubble Sort: easiest to learn, but many comparisons and swaps.")
print("- Selection Sort: fewer swaps than Bubble Sort, but still  $O(n^2)$  comparisons.")
print("- Insertion Sort: efficient for small or nearly sorted data, especially in practice.")
print()

```

output:-

Input list: [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

Bubble Sort:

Sorted list: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]

Comparisons: 45

Swaps: 24

Selection Sort:

Sorted list: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]

Comparisons: 45

Swaps: 6

Insertion Sort:

Sorted list: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]

Comparisons: 31

Shifts: 24

=== Nearly Sorted Data Test ===

Input list: [1, 2, 3, 4, 5, 10, 6, 7, 8, 9]

Bubble Sort: sorted=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], comparisons=45, movements=4

Selection Sort: sorted=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], comparisons=45, movements=4

Insertion Sort: sorted=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], comparisons=13, movements=4

AI note: Bubble Sort is simple to understand and easy to implement, but it is usually slower than the others.

Selection Sort makes fewer swaps because it selects the minimum element and swaps only once per pass.

Insertion Sort is often the best choice for nearly sorted data because it can insert items with very few shifts.

Theoretical complexity:

Bubble Sort: $O(n^2)$ time, $O(1)$ extra space

Selection Sort: $O(n^2)$ time, $O(1)$ extra space

Insertion Sort: $O(n^2)$ time in the general case, but $O(n)$ on nearly sorted data

This means the experimental results should often match the quadratic nature of these algorithms,

while Insertion Sort may perform significantly better on almost-sorted lists.

Comparison summary:

- Bubble Sort: easiest to learn, but many comparisons and swaps.
- Selection Sort: fewer swaps than Bubble Sort, but still $O(n^2)$ comparisons.
- Insertion Sort: efficient for small or nearly sorted data, especially in practice.

Task 4

Code:-

```
# Q4 - Optimising a Student Search and Sort Program

students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88)
]

# -----
# Straightforward solution
# -----

def highest_mark_simple(students_list):
    highest_name = students_list[0][0]
    highest_score = students_list[0][1]

    for name, mark in students_list[1:]:
        if mark > highest_score:
            highest_name = name
            highest_score = mark

    return highest_name, highest_score
```

```

def sort_students_by_mark_simple(students_list):
    return sorted(students_list, key=lambda student: student[1])

def find_student_by_name_simple(students_list, target_name):
    for name, mark in students_list:
        if name == target_name:
            return name, mark, True
    return None, None, False

# -----
# Improved solution
# -----

def highest_mark_optimized(students_list):
    # One pass: O(n)
    return max(students_list, key=lambda student: student[1])

def sort_students_by_mark_optimized(students_list):
    # Python's Timsort: O(n log n)
    return sorted(students_list, key=lambda student: student[1])

def build_name_index(students_list):
    # Dictionary gives constant-time lookup on average: O(1)
    return {name: mark for name, mark in students_list}

def find_student_by_name_optimized(students_list, target_name):
    student_index = build_name_index(students_list)
    if target_name in student_index:
        return target_name, student_index[target_name], True
    return None, None, False

# -----
# Demo
# -----

print("Original student list:")
print(students)
print()

# 1) Highest mark
simple_highest_name, simple_highest_mark = highest_mark_simple(students)
print("Simple highest mark:")
print(f"Student: {simple_highest_name}, Mark: {simple_highest_mark}")
print()

optimized_highest_student = highest_mark_optimized(students)
print("Optimized highest mark:")
print(f"Student: {optimized_highest_student[0]}, Mark: {optimized_highest_student[1]}")
print()

# 2) Sort by mark
simple_sorted = sort_students_by_mark_simple(students)

```

```

print("Simple sorted by mark:")
print(simple_sorted)
print()

optimized_sorted = sort_students_by_mark_optimized(students)
print("Optimized sorted by mark:")
print(optimized_sorted)
print()

# 3) Search by name
name_to_find = "Chitra"
simple_result = find_student_by_name_simple(students, name_to_find)
print("Simple name search:")
print(simple_result)
print()

optimized_result = find_student_by_name_optimized(students, name_to_find)
print("Optimized name search:")
print(optimized_result)
print()

# 4) AI notes
print("AI note: A simple solution is easy to understand, but it may repeat work unnecessarily.")
print("For example, a full loop is used to find the top mark and a second loop is used to search by name.")
print("A dictionary can avoid repeated scanning and gives much faster lookup for names.")
print()

# 5) Complexity comparison
print("Complexity comparison:")
print("Straightforward approach:")
print("- Highest mark: O(n)")
print("- Sorting by mark: O(n log n) if using a sorting routine, but a naive approach can be O(n^2)")
print("- Name search: O(n)")
print()
print("Improved approach:")
print("- Highest mark: O(n)")
print("- Sorting by mark: O(n log n)")
print("- Name search: O(1) average using a dictionary")
print()
print("Why this is better:")
print("The improved version reduces repeated scanning and uses a dictionary for direct access.")
print("This is especially useful when the program searches for many names or keeps data ready for repeated queries.")

```

output:-

Original student list:

```
[('Arun', 82), ('Bala', 65), ('Chitra', 91), ('Divya', 74), ('Esha', 88)]
```

Simple highest mark:

Student: Chitra, Mark: 91

Optimized highest mark:

Student: Chitra, Mark: 91

Simple sorted by mark:

[('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]

Optimized sorted by mark:

[('Bala', 65), ('Divya', 74), ('Arun', 82), ('Esha', 88), ('Chitra', 91)]

Simple name search:

('Chitra', 91, True)

Optimized name search:

('Chitra', 91, True)

AI note: A simple solution is easy to understand, but it may repeat work unnecessarily.

For example, a full loop is used to find the top mark and a second loop is used to search by name.

A dictionary can avoid repeated scanning and gives much faster lookup for names.

Complexity comparison:

Straightforward approach:

- Highest mark: $O(n)$
- Sorting by mark: $O(n \log n)$ if using a sorting routine, but a naive approach can be $O(n^2)$
- Name search: $O(n)$

Improved approach:

- Highest mark: $O(n)$
- Sorting by mark: $O(n \log n)$
- Name search: $O(1)$ average using a dictionary

Why this is better:

The improved version reduces repeated scanning and uses a dictionary for direct access.

This is especially useful when the program searches for many names or keeps data ready for repeated queries.

Task 5

Code:-

```
# Q5 - Optimisation Challenge: Best Combination

items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000)
]

def brute_force(items_list, budget):
    """
    Try every subset of items.
    This guarantees the optimal answer but is exponential.
    """
    best = {"total_cost": 0, "chosen": []}
    n = len(items_list)

    for mask in range(1 << n):
        total_cost = 0
        chosen = []

        for i in range(n):
```

```

        if mask & (1 << i):
            name, price = items_list[i]
            chosen.append((name, price))
            total_cost += price

    if total_cost <= budget and total_cost > best["total_cost"]:
        best = {"total_cost": total_cost, "chosen": chosen}

    return best

def greedy(items_list, budget):
    """
    Simple greedy idea:
    pick the most expensive affordable items first.
    This is fast but not always optimal.
    """
    chosen = []
    remaining = budget

    for name, price in sorted(items_list, key=lambda x: x[1], reverse=True):
        if price <= remaining:
            chosen.append((name, price))
            remaining -= price

    return {"total_cost": sum(price for _, price in chosen), "chosen": chosen}

def dynamic_programming(items_list, budget):
    """
    0/1 Knapsack.
    Guarantees the best total value without exceeding the budget.
    """
    n = len(items_list)
    dp = [[0] * (budget + 1) for _ in range(n + 1)]
    take = [[False] * (budget + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        name, price = items_list[i - 1]

        for cap in range(budget + 1):
            if price > cap:
                dp[i][cap] = dp[i - 1][cap]
            else:
                skip = dp[i - 1][cap]
                take_it = dp[i - 1][cap - price] + price

                if take_it > skip:
                    dp[i][cap] = take_it
                    take[i][cap] = True
                else:
                    dp[i][cap] = skip

    chosen = []
    cap = budget

    for i in range(n, 0, -1):
        if take[i][cap]:

```

```

        name, price = items_list[i - 1]
        chosen.append((name, price))
        cap -= price

    chosen.reverse()

    return {
        "total_cost": dp[n][budget],
        "chosen": chosen
    }

def compare_all(budget):
    brute = brute_force(items, budget)
    greedy_result = greedy(items, budget)
    dp = dynamic_programming(items, budget)

    print(f"=== Budget: {budget} ===")
    print(f"Brute Force: total={brute['total_cost']}, chosen={brute['chosen']}")
    print(f"Greedy:      total={greedy_result['total_cost']}, chosen={greedy_result['chosen']}")
    print(f"Dynamic:      total={dp['total_cost']}, chosen={dp['chosen']}")
    print(f"Optimal? {brute['total_cost'] == dp['total_cost']}")
    print("-" * 60)

# Test at least three budgets
test_budgets = [50000, 45000, 70000, 20000]

for budget in test_budgets:
    compare_all(budget)

print()
print("AI note: Brute Force checks every subset, so it is exact but very slow.")
print("Greedy is fast, but it can fail because taking the largest items first may block a better combination.")
print("Dynamic Programming is the best choice here because it considers all item-budget states and guarantees the optimal answer.")
print()

print("Time and space complexity:")
print("- Brute Force: O(2^n) time, O(1) extra space (ignoring the subset data)")
print("- Greedy: O(n log n) time due to sorting, but not guaranteed to be optimal")
print("- Dynamic Programming: O(n * B) time and O(n * B) space, where B is the budget")
print()

print("Possible optimisation for larger inputs:")
print("- Use a 1D DP array to reduce space from O(n * B) to O(B)")
print("- If the budget is very large, use a smaller effective budget or value scaling")
print("- For very large n, consider approximate algorithms or pruning if exact optimality is not required.")

```

output:-

=== Budget: 50000 ===

Brute Force: total=50000, chosen=[('Phone', 30000), ('Tablet', 20000)]

Greedy: total=50000, chosen=[('Phone', 30000), ('Tablet', 20000)]

Dynamic: total=50000, chosen=[('Phone', 30000), ('Tablet', 20000)]

Optimal? True

=== Budget: 45000 ===

Brute Force: total=45000, chosen=[('Phone', 30000), ('Watch', 10000), ('Headphones', 5000)]

Greedy: total=45000, chosen=[('Phone', 30000), ('Watch', 10000), ('Headphones', 5000)]

Dynamic: total=45000, chosen=[('Phone', 30000), ('Watch', 10000), ('Headphones', 5000)]

Optimal? True

=== Budget: 70000 ===

Brute Force: total=70000, chosen=[('Laptop', 60000), ('Watch', 10000)]

Greedy: total=70000, chosen=[('Laptop', 60000), ('Watch', 10000)]

Dynamic: total=70000, chosen=[('Laptop', 60000), ('Watch', 10000)]

Optimal? True

=== Budget: 20000 ===

Brute Force: total=20000, chosen=[('Tablet', 20000)]

Greedy: total=20000, chosen=[('Tablet', 20000)]

Dynamic: total=20000, chosen=[('Tablet', 20000)]

Optimal? True

AI note: Brute Force checks every subset, so it is exact but very slow.

Greedy is fast, but it can fail because taking the largest items first may block a better combination.

Dynamic Programming is the best choice here because it considers all item-budget states and guarantees the optimal answer.

Time and space complexity:

- Brute Force: $O(2^n)$ time, $O(1)$ extra space (ignoring the subset data)
- Greedy: $O(n \log n)$ time due to sorting, but not guaranteed to be optimal
- Dynamic Programming: $O(n * B)$ time and $O(n * B)$ space, where B is the budget

Possible optimisation for larger inputs:

- Use a 1D DP array to reduce space from $O(n * B)$ to $O(B)$
- If the budget is very large, use a smaller effective budget or value scaling
- For very large n , consider approximate algorithms or pruning if exact optimality is not required.